

PROJECT_ANALYSIS_REPORT

Hy3 数学解题与过程评估系统分析报告

1. 系统框架与面向过程评估的 Solver 设计

1.1 整体框架与职责边界

本项目将数学解答生成、最终答案验证和过程评估拆分为三个独立环节：

- 求解器（Solver）负责生成并保存可见分步解答；
- 答案验证器（Answer Verifier）独立判断最终答案；
- 过程评估器（Process Evaluator）检查可见推理、定位首个主要错误并归类错误类型。

```
Benchmark JSONL
-> Solver
-> 可见分步解答 + Final Answer
    |-> Answer Verifier (最终答案)
    `-> Process Evaluator (推理过程)
```

这种分离避免 Solver 使用参考答案自我修正或自我评分，也使同一份回答能够被不同版本的解析器和 Process Evaluator 重复分析。最终答案正确性与推理过程有效性分别由独立机制进行验证，两项指标具有不同评价含义，不能相互替代。

Answer Verifier 采用独立的确定性验证层，不依赖 Process Evaluator 或另一次 LLM 判断。

Answer Verifier 先从 Solver 可见输出中提取最终答案，再与标准答案进行精确匹配和数学等价判断，从而识别分数、小数、根式及普通代数表达式之间的等价写法；区间、多根、集合等结构化答案保留人工复核标记。

只有完整生成且成功提取答案的样本进入正常校验，截断结果统一记为未验证。因此，答案正确性与过程正确性分别由不同机制提供证据。

1.2 面向过程评估的 Solver 输出设计

本项目在 Level 1-5 各5题上对两个Solver Prompt版本进行了同题配对实验：

对比项	math-solver-v1	math-solver-v2	v2相对变化
输出约束	连续编号步骤	连续编号的数学阶段、关键依据、必要条件与独立 Final Answer	结构要求更明确
样本数	25	25	-
完整生成	25/25	25/25	不变
最终答案准确率	25/25 (100%)	25/25 (100%)	不变
Process Evaluator判定过程有效	25/25	25/25	不变
可见步骤总数	155	117	-24.5%
平均可见步骤数	6.20	4.68	-24.5%
结构问题/需复核样本	5	0	-100%

对比项	math-solver-v1	math-solver-v2	v2相对变化
Solver completion tokens	123,402	119,145	-3.5%
Solver total tokens	125,879	127,347	+1.2%
Evaluator调用次数	180	142	-21.1%
Evaluator total tokens	320,085	263,349	-17.7%

v2仍保留自然语言数学过程，不要求Solver输出JSON、步骤依赖、置信度或正确性标签，并将步骤判断、首错定位和错误分类交给独立的Process Evaluator。由于这组实验没有错误过程，对比结果只能说明v2改善了输出结构稳定性和可评估性，不能证明其提升了数学正确性或首错定位能力。

1.3 输入隔离与可见过程证据边界

Benchmark包含参考答案、参考解答、难度和学科等信息，但 Solver 输入只保留题目正文及必要样本标识，参考信息不会进入模型上下文。这一限制在数据加载层统一执行，保证答案验证和过程评估独立于生成。

Hy3响应中的 response.content 被定义为 Solver 正式公开的数学解答，也是 Process Evaluator 唯一读取的模型过程。内部 reasoning_content 只作本地运行诊断，不参与正式评分。如果关键条件或推导依据没有写入可见解答，即使模型可能在内部考虑过，也不能作为过程成立的证据。

1.4 生成完整性与实验记录

鉴于长推理输出可能受到生成长度限制，实验协议将 API 请求成功与模型生成完整性作为两个独立状态记录。只有 finish_reason=stop 的结果被视为完整生成并进入正常答案验证；截断结果保留用于审查，但不计为模型答案错误。

Solver 使用确定性Parser提取 Step 和最终答案候选，不修改原始回答，也不判断数学正确性。每次生成保留Prompt版本、请求参数、完成状态、可见回答、解析结果和用量，使答案验证、过程评估及人工复核能够关联到同一份冻结解答。

2. Process Evaluator 方法设计

2.1 标准化 Step 输入

Process Evaluator 接收 Solver 生成的可见 Step 序列和独立 Final Answer 。评估流程如下：

```
Solver response.content
  -> Step Parser
  -> 局部评估器 (Local Evaluator)
  -> 首错候选定位
  -> 全局评估器 (Global Evaluator)
  -> 确定性聚合器 (Deterministic Aggregator)
```

本项目没有采用单次 LLM 调用直接输出过程正确性、首错位置和错误类型，而是将任务拆分为 Local Evaluator 逐步评估、Global Evaluator 整体检查和 Deterministic Aggregator 确定性聚合。

分阶段设计能够分别保留步骤级判断依据与全局覆盖信息，避免一次整体判断中后续正确结论掩盖前序错误，并减少最终汇总阶段再次引入生成式不确定性。该设计增加了一定的调用次数

和重复上下文开销，但能够提供更明确的错误来源信息和可复查的中间判断结果。

步骤解析器（Step Parser）只保留步骤编号、步骤原文和结构问题，不补写缺失步骤、不修改数学表达，也不判断正确性。评估粒度与 Solver v2一致：一个 Step 表示相对完整的数学阶段，可以包含紧密相关的短推导；首错以 Step N 为单位，不延伸到步骤内部的单个符号。

Local Evaluator 和 Global Evaluator 均使用固定的结构化输出 schema 记录步骤状态、重要性、错误来源、错误类型、首错候选及全局判断。只有字段完整、枚举合法且 Step 编号有效的结果才能进入 Deterministic Aggregator；不符合 schema 的响应不会通过自由文本推断或静默修复，从而减少 LLM 输出格式波动带来的解析歧义。

2.2 Local Evaluator：逐步判断当前推理

第 i 次 Local Evaluator 调用只读取完整题目、此前已经出现的步骤和当前 Step，不读取后续步骤。这样设计一方面可以按解答本身的推理顺序判断当前结论是否由已有证据支持，另一方面可以避免后文给出的正确结果反向掩盖前面的错误或证据缺口。

各 Step 通过相互独立的 Local Evaluator 请求进行评估，不继承前一调用的自由文本推理历史。每次请求中的前序步骤均来自原始 Solver 可见解答，而不是前一 Local Evaluator 调用生成的判断内容，从而降低前序评估结果向后续步骤判断传播的依赖风险。

Local Evaluator 为每一步输出四种状态：

状态	含义
valid	当前推导或计算成立，且可见文本提供了足够依据
invalid	当前步骤存在明确数学错误
insufficient	当前结论可能正确，但缺少确认它所需的关键可见依据
uncertain	现有信息不足以可靠归入前三种状态

区分 invalid 和 insufficient 可以避免把结论错误与结论可能正确但证明不足混为一类。例如，具体计算出错属于 invalid；得到正确最大值却没有说明其为何达到全局最大，则属于 insufficient。uncertain 用于保留真正无法可靠裁决的情况，使系统可以进入人工复核，而不是强制给出二元判断。

除状态外，Local Evaluator 还记录当前步骤的重要性以及错误来源。错误来源分为当前步骤引入、继承前面错误、无错误和无法确定。该字段用于区分首个错误与沿用错误结果的后续步骤：如果后续步骤只是基于先前错误数值进行了局部上正确的计算，它应被标记为继承错误，而不应再次成为新的首错。

2.3 首错定位：寻找最早的主要错误事件

首错定位不是简单选择第一个非 valid 步骤。聚合规则优先选择最早由当前步骤引入的明确错误，或首个会影响结论的高重要性证据缺口；继承错误和低影响的说明缺失不会自动成为首错。

数学解答中的后续错误往往只是早期错误的传播结果。如果把所有错误步骤并列统计，系统会重复计算同一错误来源，也无法准确确定解答从何处开始失效。因此，项目先定位最早主要错误事件，再基于该事件选择错误类型，后续症状只作为传播证据保留。

2.4 Global Evaluator：补足跨步骤与最终结论检查

Local Evaluator 适合发现当前步骤中的计算、概念和具体推导错误，但仅靠逐步判断难以稳定识别跨步骤问题。例如，一个解答可能每个已写出的分支都计算正确，却遗漏必要分支；也可能前面得到正确数值，最终却输出了错误选项。为此，Global Evaluator 读取题目、完整可见解答和已经通过格式校验的 Local Evaluator 结果，进一步检查：

- 是否遗漏必要条件、候选解或分类讨论；
- 是否存在跨步骤跳跃、循环论证或前后不一致；
- 是否完整回答题目要求；
- 最终答案是否真正受到可见过程支持；
- Local Evaluator 候选首错是否需要被全局证据修正。

final_answer_supported 采用严格含义：只有可见过程数学上有效、依据充分并能够推出最终答案时才为 true 。一个错误答案即使与一条错误推导链条保持形式一致，也不构成数学支持；一个正确答案如果来自无效推导、关键条件遗漏或无法解释的跳转，同样应为 false 。

Local Evaluator 和 Global Evaluator 形成互补关系。

Local Evaluator 保留按推理顺序展开的逐步证据，Global Evaluator 负责检查只有看到完整解答后才能判断的覆盖性和结论支持度。两者不采用互相覆盖的简单优先级：如果 Global Evaluator 给出的首错与 Local Evaluator 候选首错冲突，Deterministic Aggregator 保留冲突、取消确定首错并要求人工复核。

2.5 Deterministic Aggregator：确定性聚合与答案—过程关系

Local Evaluator 和 Global Evaluator 均由 Hy3 完成判断，但最终结果由不使用 LLM 的 Deterministic Aggregator 生成。Deterministic Aggregator 综合逐步状态、错误来源、Global Evaluator 完整性、最终答案支持度和独立答案验证结果，输出过程正确性、首错位置、错误类型、是否需要复核以及答案—过程关系。

答案正确性只在聚合阶段读取，不会提供给 Local Evaluator 或 Global Evaluator 。因此，Process Evaluator 不能根据最终答案与参考答案一致倒推中间步骤正确，也不能因为最终答案错误就跳过过程检查。聚合结果可以明确表示答案正确但过程无效；确定性规则也避免再让 LLM 自由改写已有判断，使最终决策可以复查和重复执行。

3. 错误分类体系与受控错误设计

3.1 分类目标与设计原则

错误分类体系用于描述导致解答首次实质失效的主要原因，而不是为每个后续错误现象都分配标签。设计时遵循三个顺序：先定位最早主要错误事件，再排除继承错误和下游症状，最后依据定义和排他边界选择唯一类型。

当前体系包含8个主要类型和1个兜底类型：

错误类型	核心含义	与相邻类型的边界
problem_misinterpretation	错误理解任务、已知信息、所求对象或操作顺序	问题出在对题意的表示，而非后续计算执行
condition_omission	没有检查定义域、符号、边界、非零性、整数性、不同性或定理适用条件	已有候选但未执行可接受性限制
case_omission	没有覆盖必要分支、候选根或解族	缺少替代分支或候选本身，而非筛选条件
concept_or_theorem_error	陈述或依赖了错误的一般概念、公式、恒等式或定理	错误发生在一般规则层面
invalid_derivation	当前具体结论无法由可见前提推出	具体变换不成立，但不一定陈述了错误的一般规则

错误类型	核心含义	与相邻类型的边界
calculation_error	运算形式有效，但具体计算、化简、代入、符号或抄写执行错误	方法本身成立，错误在一次具体执行
insufficient_justification	未发现确定错误或明确遗漏，但关键结论缺少必要依据	结论可能正确，只是可见证据不足
answer_extraction_or_format_error	数学过程正确且充分，唯一错误位于最终答案的选择、抄写或格式	只有前述过程有效时才能使用
other	已确认存在问题，但无法归入上述类型	仅作兜底，不用于主动扩大类别

实验过程中出现的最容易混淆的边界包括：遗漏候选属于 `case_omission`，没有用条件排除不可接受候选属于 `condition_omission`；错误的一般公式属于 `concept_or_theorem_error`，某次具体推导不成立属于 `invalid_derivation`，合法方法中的具体算错属于 `calculation_error`。如果非法变换首先导致解丢失，首要类型应是该处 `invalid_derivation`，而不是把后续少解重新标为 `case_omission`。

3.2 受控错误的人工设计与注入

常规解答验证集中仅出现1例过程错误，无法覆盖分类体系。项目因此从16道不同的 `Solver v2` 正确解答出发，每例只注入一个预先定义的主要错误，并尽量保留原有 `Step` 边界及后续传播过程。注入后的解答均作为人工构造的受控样本单独标识，不计入 `Solver` 的常规生成结果。

受控错误验证集覆盖 `MATH` 的 `Level 1–5`，其中 `Level 1–3` 各2例，`Level 4–5` 各5例。

16例中有13例同时得到错误答案，另有3例保留正确答案但使过程错误或依据不足，以验证答案—过程独立性。为避免受控错误样本包含直接揭示错误位置或类型的文本线索，构造过程中移除了“这里忽略了”“该步骤错误”等自我揭示性表述，使错误以正常的数学陈述、计算或选择形式出现。

按当前v1.2人工复核标签，16例构成为：

类型	案例数
<code>concept_or_theorem_error</code>	4
<code>condition_omission</code>	3
<code>calculation_error</code>	2
<code>case_omission</code>	2
<code>invalid_derivation</code>	2
<code>problem_misinterpretation</code>	1
<code>insufficient_justification</code>	1
<code>answer_extraction_or_format_error</code>	1
合计	16

3.3 分类 Prompt 的修订

为了给LLM提供错误类型分类依据，该项目还设计了不同错误类型的提示。

早期分类 Prompt 只给出了错误类型名称和简短定义，分类结果在 `concept_or_theorem_error`、`invalid_derivation` 和 `calculation_error` 等相邻类别之间存在较明显的归类不稳定现象。

改进后的错误类型 v1.2 分类 Prompt 增加了排他边界和诊断问题，并明确要求先定位首错、再排除继承错误、最后分类。人工标签也按同一版定义重新复核，以避免用新版预测直接对比旧版分类口径。

新版 high 配置的 Process Evaluator 在16例中实现错误类型准确率14/16（87.5%）。剩余偏差集中在相邻类别：不等式关系未保持被判为计算错误而非非法推导；未执行正半径条件被判为答案提取错误而非条件遗漏。两例均正确检出并定位问题，差异发生在错误归因层面。

4. Process Evaluator 有效性验证

4.1 验证任务与证据

参考答案只能验证最终结论，参考解答也只是可能的正确方法之一，不能直接提供逐步状态、首错位置和错误类型。项目使用两类互补材料验证 Process Evaluator：

验证材料	数量	主要目标
受控错误验证集	16	验证错误检出、首错定位和类型分类
常规解答验证集	45	通过逐题人工复核检查过程判定和误报

受控错误验证集为多种已知错误提供明确参考标签；常规解答验证集用于检验 Process Evaluator 能否接受有效过程并发现常规生成解答中的过程问题。两组验证实验承担不同任务，不合并成总体准确率。

4.2 主要有效性结果

验证指标	Process Evaluator（high）
错误检出率	16/16（100%）
首错定位准确率	15/16（93.8%）
错误类型准确率	14/16（87.5%）
整体过程判定一致率	16/16（100%）
最终答案支持关系一致率	16/16（100%）

常规解答验证指标	结果
人工确认过程正确	44
人工确认过程错误	1
过程正确样本误报	0/44（0%）
过程错误样本正确检出	1/1（100%）
Process Evaluator 与人工判断一致	45/45（100%）

常规解答验证集中的45份由 low 配置 Solver 生成的解答均经过逐题复核。当前验证集中未观察到误报，但仅包含1例人工确认的过程错误样本，不能据此推断更广分布上的稳定误报率或漏报率。

首错定位采用严格口径：最终答案格式错误案例没有具体错误Step，即使Evaluator也输出空步骤，仍按未准确定位计算。这些结果说明当前 Process Evaluator 在小规模验证范围内能够

识别过程错误、定位首错并接受有效过程；受控错误验证集规模和人工设计分布仍限制了结论外推。

5. 典型案例分析

5.1 常规解答中的答案正确但过程错误

常规解答验证集中的 `math-test-prec calculus-0488` 要求计算四个两两外切圆中最小圆的半径。low 配置的 Solver 最终得到正确答案 $6/23$ ，但 Step 7 把前一步等式重排为：

$$0 = 1 + 11r/3 + 23r^2/36$$

正确重排后的常数项应为 -1 。可见解答下一行又直接使用正确的二次方程 $23r^2 + 132r - 36 = 0$ ，但没有给出从错误等式切换到正确方程的有效过渡。Local Evaluator 和 Global Evaluator 均将 Step 7 判为 `calculation_error`，聚合结果为 `correct_answer_invalid_process` 且 `final_answer_supported=false`。人工复核确认这是可见过程中的真实错误。

该案例说明最终答案准确率不能代替过程正确率。若只比较答案，该样本会被统计为正确；加入过程评估后，可以发现其可见数学推导无法连续支持最终结论。

5.2 正确结论但缺少关键证明

Level 3 最大面积案例保留正确的50乘50构造和最终答案2500，但删除了证明该构造达到最大值的关键依据。这里没有明确错误计算，问题是可见过程不足以确认全局最大性，因此人工状态为 `insufficient`，类型为 `insufficient_justification`。

high 配置的 Process Evaluator 正确识别该证据缺口并给出一致类型。low 配置仍定位到相同步骤并识别相同类型，但没有形成确定的整体错误判定，而是进入复核状态。该案例体现了四态设计的必要性：关键依据不足不应被归为并不存在的计算错误，同时其重要性又足以阻止系统直接接受整个过程。

5.3 分支遗漏与首错生效位置

Level 5 复数方程案例先分析 $y=0$ 分支，再在 Step 5 声称只有该分支的解，从而遗漏 $x=-1$ 分支。人工首错标在 Step 5 而不是 Step 4：单独分析一个合法分支本身并没有错，直到声称解已经穷尽时，遗漏才成为实质错误。

high 配置的 Process Evaluator 正确定位 Step 5 并分类为 `case_omission`。low 配置能够识别相同类型，但整体判断不确定，并把首错提前到 Step 4。这个差异说明首错定位不仅要求发现缺少哪个分支，还要求判断遗漏在推理流程中的实际生效时点。

5.4 错误规则偶然得到正确结论

Level 5 数论受控案例使用了错误的一般质因数指数规则，但由于当前具体指数为1，错误论证偶然得到正确中间结论并最终保留正确答案42。high 配置的 Process Evaluator 识别 Step 3 的 `concept_or_theorem_error`，并判定为正确答案但错误过程；low 配置则没有检出该错误，最终把过程判为有效。

该案例要求 Process Evaluator 区分最终结论正确性与支撑该结论的一般规则有效性，因此仅检查数值结果是否一致不足以完成过程判定。

low 配置未检出该错误，表明降低推理强度后，这类最终结论正确但推理依据无效的问题更容易被遗漏。

6. Solver 与 Process Evaluator 能力边界分析

考虑到 Hy3 推理调用，特别是高推理强度和多阶段过程评估，会产生较高的 API token 开销，本项目未对已构建的完整候选题集进行全量调用，而是在保证难度分层和主要验证目标覆盖的前提下采用受控规模的实验样本。

因此，本文实验主要用于验证系统设计和比较不同推理配置，不以大规模统计估计 Hy3 的总体数学能力为目标。

6.1 对照实验设计

本项目中的推理强度影响两个不同任务：Solver 负责生成数学解答，Process Evaluator 负责检查冻结的可见解答。若同时改变两者，就无法判断结果差异来自解题过程变化还是评估判断变化。因此，能力边界分析采用两组独立对照：

分析对象	保持不变	改变变量	回答的问题
Solver	同一45题、Prompt v2、生成参数和答案验证方法	Solver 的 high 与 low 配置	解题结果和生成成本是否变化
Process Evaluator	同一16份受控解答、Step 划分、人工标签、评估Prompt和 schema	Process Evaluator 的 high 与 low 配置	评估能力和评估成本是否变化

这种拆分避免把整条流水线的变化归因于单一组件。早期25题实验采用的 Process Evaluator 评估协议与当前版本不同，因此不纳入 high 与 low 配置的直接定量对照。

6.2 Solver：推理成本与解题表现

在完全相同的45道 MATH 纯文字题上，high 与 low 配置的 Solver 完整生成率和最终答案准确率均为45/45（100%）。成本差异如下：

指标	high	low	变化
总 token 用量	311,089	113,594	-63.5%
推理 token 用量	275,718	79,193	-71.3%

在当前样本中，low 配置显著减少内部推理开销，但未观察到最终答案准确率下降。常规解答验证集的人工复核结果为44/45（97.8%）过程有效，唯一无效样本位于 Level 5，且属于答案正确但 Step 7 计算符号不一致。

这些结果尚不能证明 low 与 high 具有相同的数学能力。首先，high 解答没有与 low 完全一致的45题人工过程标签，因此不能进行严格的过程正确率比较；其次，Level 1-3 各只有5题，Level 4-5 各15题，样本量不足以确定难度临界点；再次，两种配置均在 temperature=0.9 下每题单次生成，结果不能代表重复运行后的稳定期望。

因此，Solver 侧目前能够支持的结论是：在这45道同题单次生成中，降低推理强度带来约三分之二的 token 节省，但未观察到最终答案性能下降。项目尚未确定 Hy3 是否从 MATH 的某一 Level 开始出现明显的能力下降。

6.3 Process Evaluator：推理预算敏感性

Process Evaluator 的 high 与 low 对照使用同一16份冻结受控错误解答，唯一变化是推理强度：

指标	high	low
错误检出率	16/16（100%）	15/16（93.8%）

指标	high	low
首错定位准确率	15/16 (93.8%)	13/16 (81.3%)
错误类型准确率	14/16 (87.5%)	14/16 (87.5%)
整体过程判定一致率	16/16 (100%)	13/16 (81.3%)
needs_review	1/16 (6.3%)	3/16 (18.8%)
总 token 用量	353,599	290,652
推理 token 用量	134,182	76,315

low 相对 high 减少43.1%的推理 token 和17.8%的总 token 用量，但错误检出率由100%降至93.8%，严格首错定位准确率由93.8%降至81.3%，整体过程判定一致率由100%降至81.3%。错误检出按Local或Global是否给出明确的 invalid 或 insufficient 证据统计，不要求聚合结果必须为 process_correct=false 。因此，low能够发现15例存在问题，但其中部分未能准确定位或形成确定的整体判定。两种配置的错误类型准确率虽然均为87.5%，但失败案例并不一致。

low 的主要失败包括：

- 已发现关键最大性依据缺失，但没有形成确定的整体过程判定；
- 已发现复数分支遗漏，但首错位置被提前且整体判断不确定；
- 错误质因数指数规则被完全漏检。

当评估任务需要区分最终结论正确与推理依据有效这两种情况，或需要判断遗漏在何时真正生效时，较低推理强度更容易产生漏检与不稳定定位。

Process Evaluator 在 high 与 low 配置之间已经表现出比 Solver 更明显的推理强度敏感性。Process Evaluator 需要重建推理依赖、区分错误源头与下游症状，并检查结论是否由正确依据支持，因此对推理预算比当前 Solver 解题样本更加敏感。

因此，当前实验中更明确的能力边界并非 MATH 某一 Level 上的 Solver 准确率下降，而是生成任务与过程审查任务对推理预算表现出不同敏感性。Solver 在当前样本下对推理强度下降相对稳定，Process Evaluator 则出现明显的错误检出与首错定位退化。项目采用 high 作为 Process Evaluator 主要验证配置，low 用于展示成本与评估能力之间的trade-off。

6.4 实验边界与局限

现有实验能够说明：

- 最终答案正确不保证可见过程有效，常规解答验证集和受控错误验证集均包含答案正确但过程错误的样本；
- Solver 的 low 配置在当前45题上显著降低成本，但尚未观察到最终答案准确率下降；
- Process Evaluator 的 low 配置在冻结受控错误上出现错误漏检、首错提前和更多复核，high 更适合作为当前主要评估配置；
- Local Evaluator 、Global Evaluator 和 Deterministic Aggregator 分别承担逐步判断、整体覆盖检查和答案—过程关系合并，具有不同且互补的作用。

受 API 调用额度与实验成本限制，主实验和有效性验证均采用有限样本规模。虽然样本覆盖 MATH 的多个难度层级，并针对错误类型进行了受控覆盖，但现有规模不足以支持对细粒度难度趋势、稳定性能临界点和自然错误分布进行可靠统计推断。